CY Cergy-Paris Université

RAPPORT DE PROJET

pour le projet de l'UE Génie Logiciel
Licence 2 informatique, Année 2024 - 2025

sur le sujet

Chess

rédigé par

Adel MAHI, Bilel BEN NAYA, Aymen ESSAHLI



Avril 2025

CERGY PARIS UNIVERSITÉ

Table des matières

1 Introduction **4**
1.1 Contexte du projet . . . 4
1.2 Objectif du projet . . . 4
1.3 Organisation du rapport . . . 4

2 Spécification du projet **6**
2.1 Notions de base et contraintes du projet . . . 6
2.1.1 Fonctionnement général du logiciel . . . 6
2.1.2 Notions et terminologies de base . . . 6
2.1.3 Contraintes et limitations connues . . . 7
2.2 Fonctionnalités attendues du projet . . . 7

3 Conception et mise en œuvre **9**
3.1 Architecture globale du logiciel . . . 9
3.1.1 Diagramme de cas d'utilisation . . . 9
3.2 Classes de données . . . 10
3.3 IHM Graphique . . . 10
3.3.1 Conception IHM graphique . . . 10
3.3.2 Conception technique de l'IHM graphique . . . 12
3.4 Conception des traitements (processus) . . . 12
3.4.1 Traitement du déplacement d'une pièce . . . 12
3.4.2 Validation spécifique des mouvements . . . 14
3.4.3 Traitement du calcul du meilleur coup . . . 15
3.4.4 Gestion de la capture d'une pièce . . . 16
3.4.5 Détection de la fin de partie . . . 17

4 Manuel utilisateur **18**
4.1 Démarrage et choix du mode de jeu . . . 18
4.2 Plateau de jeu . . . 18
4.3 Sélectionner et déplacer une pièce . . . 19
4.4 Historique des coups . . . 20
4.5 Fin de la partie . . . 21

5 Déroulement du projet **23**
5.1 Réalisation du projet par étapes . . . 23
5.2 Répartition des tâches, communication et intégration . . . 23

6 Conclusion et perspectives **25**
6.1 Résumé du travail réalisé . . . 25
6.2 Améliorations possibles du projet . . . 25

Table des figures

1 Première conception du plateau . . . 6
2 Diagramme de cas d'utilisation . . . 9
3 Diagramme de classes de données . . . 10
4 Fenêtre de choix de mode de jeu . . . 11
5 Fenêtre de jeu . . . 11
6 Fenêtre d'accueil du logiciel . . . 18
7 Plateau de Xiangqi à l'initialisation . . . 19
8 Illustration de mouvement valide . . . 20
9 Affichage de l'historique des coups . . . 21

10 Message de fin de partie 22
11 Diagramme de Gantt 23

Liste des tableaux

1 Fonctionnalités principales 8

Remerciements

Nous tenons à remercier vivement toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail. Leur soutien, leurs conseils et leur bienveillance ont grandement participé à la réussite de ce projet et à l'enrichissement de notre expérience d'apprentissage.

Nous souhaitons tout particulièrement exprimer notre profonde gratitude à notre encadrant, Monsieur Tianxiao Liu, pour son suivi attentif, ses conseils avisés et sa disponibilité constante tout au long de ce projet. Sa rigueur, son expertise dans le domaine ainsi que sa capacité à nous guider tout en nous laissant une marge d'autonomie ont été des éléments déterminants pour la bonne conduite de notre travail. Ses retours pertinents et son exigence de qualité nous ont permis de progresser à chaque étape du projet. Ce travail est le fruit d'un véritable effort collectif et collaboratif dont nous sommes fiers.

1 Introduction

1.1 Contexte du projet

Pour la fin du semestre 4, nous avions pour objectif de réaliser une adaptation du jeu d'échecs chinois : le Xiangqi. Dans ce cadre, il nous a été demandé de choisir un sujet parmi une liste de 50 projets proposés par M. Tianxiao Liu. Après mûre réflexion et discussion au sein de notre équipe, notre choix s'est naturellement porté sur le jeu de Chess.

Les raisons de ce choix sont multiples. Avant tout, nous, membres de ce groupe, portons un réel intérêt pour les mécanismes de stratégie et de planification, des compétences que l'on retrouve notamment dans des jeux comme le Xiangqi ou encore le jeu d'échecs occidental. Parmi tous les projets proposés, celui de Chess représentait pour nous l'option qui se rapprochait le plus de notre affinité avec les jeux de réflexion, où la prise de décision, la prévision des actions adverses et la gestion des ressources sont primordiales.

De plus, dès l'annonce de ce sujet, nous avions déjà en tête plusieurs idées concernant sa conception ainsi que les différentes fonctionnalités et rendus possibles à atteindre. Cette prévisualisation du projet final nous a motivés à le placer parmi nos premiers choix.

Enfin, nous avons estimé que ce projet serait le plus adapté aux forces de notre équipe. En effet, chacun des membres possède des compétences spécifiques qui, mises en commun, permettaient d'aborder ce projet avec efficacité et ambition. Le développement d'un tel jeu nécessitait à la fois une bonne maîtrise des aspects graphiques, de la logique de jeu et de la gestion de projet, autant de domaines dans lesquels nos membres pouvaient pleinement s'exprimer.

1.2 Objectif du projet

L'objectif principal de ce projet est de concevoir une application logicielle professionnelle, répondant à un cahier des charges précis et respectant l'ensemble des critères imposés.

À travers ce projet, il s'agissait pour nous non seulement de produire un logiciel fonctionnel, mais également d'acquérir une expérience complète et professionnalisante dans le domaine du développement informatique. Cela passe par plusieurs compétences clés : la capacité à s'organiser efficacement en équipe, à répartir les tâches de manière équilibrée, à concevoir une architecture technique robuste, à mettre en œuvre les différentes fonctionnalités prévues et enfin à présenter et défendre notre réalisation lors de la démonstration finale.

Au-delà des aspects purement techniques, ce projet nous a également permis de renforcer nos compétences transversales : communication entre membres de l'équipe, gestion du temps, adaptation aux imprévus et à l'évolution des besoins. Il constitue donc une étape importante dans notre parcours académique, en nous préparant à mieux appréhender des projets professionnels complexes à l'avenir.

1.3 Organisation du rapport

Dans ce rapport, nous proposons une explication détaillée du projet sous tous ses aspects, depuis sa conception jusqu'à sa réalisation finale.

Dans un premier temps, nous commencerons par exposer les spécifications du projet, en présentant les bases du jeu choisi ainsi que l'ensemble des contraintes fonctionnelles et techniques qui nous ont été imposées. Nous décrirons aussi notre interprétation de ces spécifications et les choix que nous avons faits pour les respecter tout en garantissant une expérience utilisateur agréable.

Ensuite, nous aborderons la phase de conception, en détaillant l'architecture du projet, la structuration du code, les différentes technologies utilisées ainsi que nos choix de conception majeurs. Nous montrerons comment nous avons cherché à produire un code modulaire, maintenable et évolutif.

Par la suite, nous reviendrons sur le déroulement du projet : l'organisation du travail au sein de l'équipe, la méthodologie adoptée (réunions, outils de collaboration, suivi de l'avancement) et la répartition des tâches entre les membres. Nous mettrons en lumière les difficultés rencontrées ainsi que les solutions que nous avons mises en place pour les surmonter.

Enfin, nous conclurons ce rapport par un bilan personnel et collectif de cette expérience, en tirant les enseignements principaux que nous en avons retirés.

2 Spécification du projet

Nous avons présenté l'objectif du projet dans la section 1. Dans cette section, nous détaillerons la spécification de notre logiciel réalisé. Ceci correspond principalement au document de spécification du projet (cahier des charges).

2.1 Notions de base et contraintes du projet

2.1.1 Fonctionnement général du logiciel

Le logiciel développé permet de simuler une partie de Xiangqi (échecs chinois) en respectant l'ensemble des règles du jeu. Lors du lancement du programme, une fenêtre graphique (interface Swing en Java) s'ouvre, affichant un plateau de Xiangqi de 11 colonnes par 11 lignes avec les pièces positionnées selon leur configuration initiale.

2.1.2 Notions et terminologies de base

Règles de base du jeu de Xiangqi

Le Xiangqi, ou échecs chinois, est un jeu de stratégie se jouant sur un plateau de 9 colonnes par 10 rangées, divisé par une rivière centrale. Chaque joueur dispose de 16 pièces avec des rôles spécifiques, dont le général, les chars, les chevaux, et les canons et chacune de ces pièces sont placées sur les intersections de cases.

Le plateau de Xiangqi, de base, est composé de 9 colonnes et 10 rangées ainsi qu'une « rivière » (ligne de cases vides) séparant les deux camps. De plus, chaque camp dispose d'un « palais » de 3 cases par 3 cases au centre de sa dernière rangée, ou le roi est confiné. Chaque pièce a un mode de déplacement qui lui est propre et peut en « capturer » une autre en se déplaçant sur la case occupée par une pièce ennemie. Pour gagner une partie, l'objectif est de mettre le roi adverse en échec et mat, c'est-à-dire qu'il ne puisse plus échapper à une capture. De plus si les rois des deux équipes respectives sont sur la même colonne et qu'aucune pièce ne les sépare, la partie se termine.

Voici la figure 7 conceptuelle de base de notre plateau qui permetttra de visualiser l'emplacement des pièces.

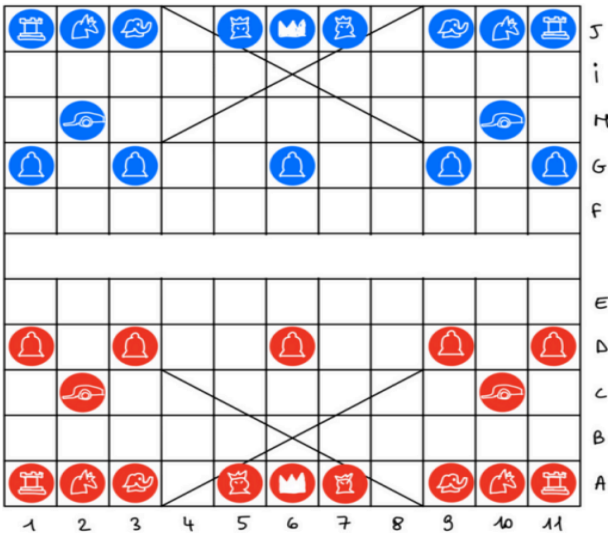


FIGURE 1 – Première conception du plateau

Règles des pièces :

- Général (ou roi) : Il sera placé en A6. Il se déplace d'une case à la fois horizontalement ou verticalement et doit rester dans son palais sans jamais être face à l'autre général.
- 2 Gardes : Les gardes sont placés à la droite et la gauche du général (en A5 et A7). Ils se déplacent en diagonale d'une case et ne peuvent pas quitter le palais.
- 2 Éléphants : Ils seront placés en A3 et en A9. Ils se déplacent de deux cases en diagonale, ne peuvent pas traverser la rivière et si une pièce se trouve entre sa position de départ et sa destination, le déplacement est interdit. Dans le cadre de ce projet (et des modifications prévu pour l'adaptation du Xiangqi), les éléphants auront un déplacement légèrement différent :
  - L'éléphant se trouvant en A3 pourra se déplacer en C1, E3 et C6.
  - L'éléphant se trouvant en A9 pourra se déplacer en C11, E9 et C6.
- 2 Chars (ou tours) : Ils seront placés en A1 et A11. Les chars se déplacent horizontalement ou verticalement sur n'importe quelle distance sans obstacle.
- 2 Chevaux : Ils seront placés en A2 et A10. Les chevaux se déplacent en "L" (c'est-à-dire de deux cases (horizontalement ou verticalement) puis d'une case (verticalement ou horizontalement), mais leur mouvement est bloqué s'il y a une pièce adjacente sur leur trajectoire.
- 2 Canons : Les canons seront placés en C2 et en C10. Ils se déplacent comme les chars (horizontalement ou verticalement) mais pour capturer une pièce, le canon doit "sauter" au-dessus exactement une pièce
- 5 Soldats : Ils seront placés en D1, D3, D6, D9 et D11. Avant la rivière, ils ne peuvent avancer que d'une case verticalement, mais après l'avoir traversée, ils peuvent aussi se déplacer latéralement.

2.1.3 Contraintes et limitations connues

Pour rappel, notre projet vise à réaliser une **adaptation** du jeu de Xiangqi. Ainsi, en fonctions des contraintes de notre projet et limitations connues, nous devrons modifier quelques éléments du jeu tout en gardant l'essence même du jeu. Ces adaptations de règles ou concernant l'interface permettront une meilleur utilisation pour l'utilisateur tout en préservant l'esprit du jeu.

Modifications :

- Le plateau ne contiendra plus 9 colonnes et 10 rangées mais 11 colonnes et 11 rangées.
- Deux colonnes seront ajoutées respectivement entre les gardes et les éléphants.
- La taille du palais sera désormais de 5 cases (horizontalement) par 3 cases (verticalement).
- En tenant compte des modifications précédentes, nous adapterons donc les déplacement des éléphants et des gardes pour que le jeu garde de la cohérence.

Outils de développement :

1. Langage de programmation : Java
2. IDE : Eclipse
3. Outil de rédaction : Latex

2.2 Fonctionnalités attendues du projet

Comme illustré dans le tableau 1, le projet est articulé autour de six grandes fonctionnalités principales. Chaque fonctionnalité contribue à offrir une expérience de jeu complète, immersive et fidèle aux règles traditionnelles du Xiangqi.

L'interface graphique constitue le cœur de l'interaction avec le joueur, en permettant de visualiser en temps réel l'état du plateau et les déplacements des pièces. La validation rigoureuse des mouvements assure le respect des règles spécifiques au Xiangqi, garantissant une partie juste et sans erreurs. La possibilité de choisir entre jouer contre un autre joueur ou contre une intelligence artificielle enrichit l'expérience de jeu.

Fonction	Coefficient	Commentaire
Choisir le mode de jeu	5%	Jouer contre l'ordinateur ou contre un autre joueur.
Interface graphique	25%	Visualiser le plateau et les pièces en temps réel grâce à une interface graphique intuitive.
Sauvegarde	10%	Le joueur doit pouvoir sauvegarder et charger une partie en cours.
Validation de jeu	40%	Effectuer des mouvements légaux selon les règles du Xiangqi adaptées. Le système doit vérifier **la validité des mouvements.**
Aide	5%	Consulter les règles du jeu dans une section d'aide intégrée.
Fin de partie	15%	Voir une notification de fin de partie lorsqu'un roi est en échec et mat ou si aucun mouvement légal n'est possible.

TABLE 1 – Fonctionnalités principales

Des fonctionnalités complémentaires telles que la sauvegarde de partie et l'aide intégrée permettent d'améliorer l'accessibilité et la fluidité d'utilisation. Enfin, un système clair de gestion de la fin de partie offre une conclusion automatique aux parties selon les conditions de victoire officielles (capture du général adverse ou impossibilité de mouvement).

3 Conception et mise en œuvre

C'est une longue section, les sous-sections contenant les sous-sous-sections sont acceptables.

3.1 Architecture globale du logiciel

3.1.1 Diagramme de cas d'utilisation

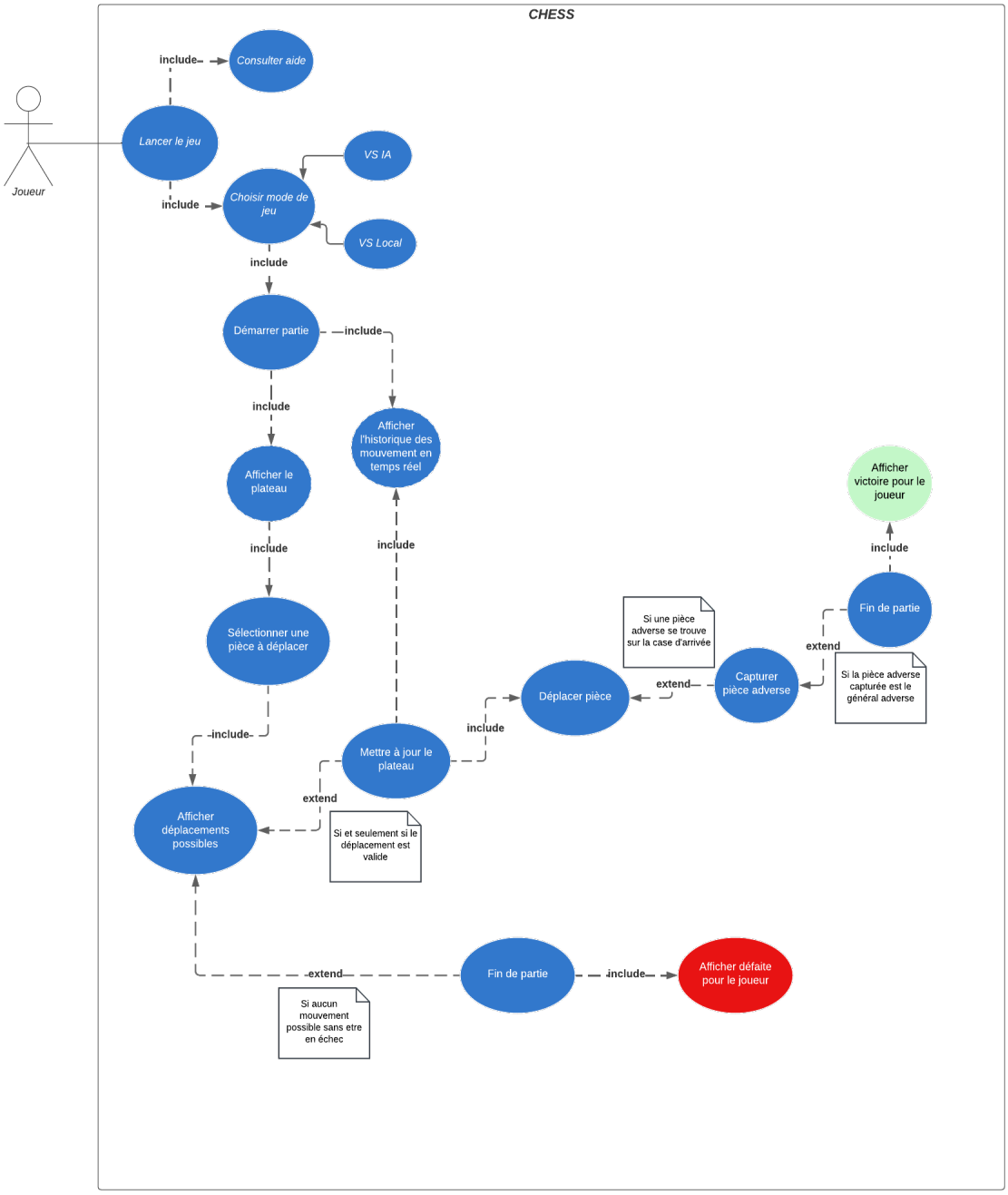


FIGURE 2 – Diagramme de cas d'utilisation

Dans la figure 2, nous pouvons voir tout le déroulé des différentes actions que le joueur pourra réaliser du commencement de la partie à la fin de la partie.

3.2 Classes de données

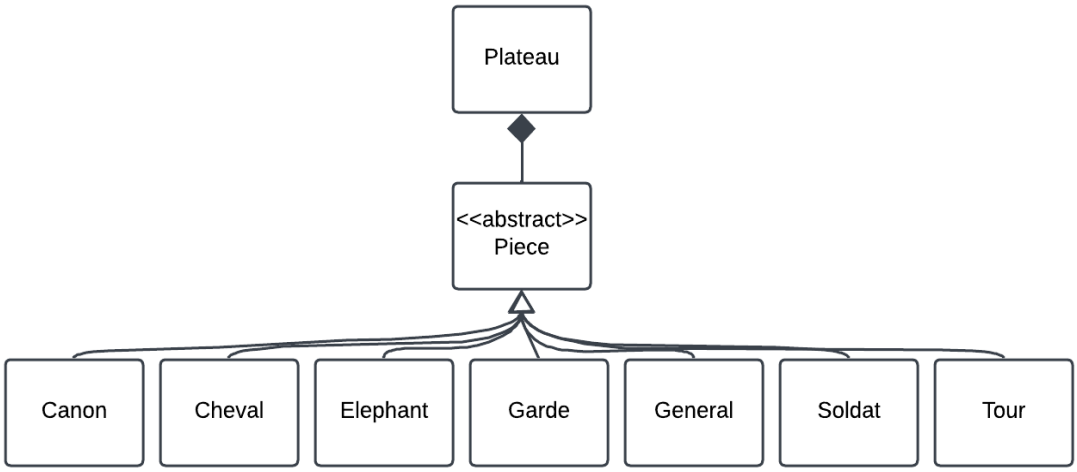


FIGURE 3 – Diagramme de classes de données

Pour les classes de données, nous avons décider d'opter pour une structure simple et intuitive afin de faciliter l'implémentation du jeu et son évolution éventuelle.

Tout d'abord, la classe **Plateau** permet de définir la structure du jeu. Elle contient un tableau de pièces représentant l'état du plateau à chaque instant de la partie. Ce tableau 11 x 11 est constituée d'objets héritant de la classe abstraite **Piece**. Chaque case du plateau peut ainsi soit contenir une pièce (Général, Garde, Éléphant, Cheval, Canon, Tour, Soldat), soit être vide. Cette classe permet aussi de gérer le déroulement du jeu : elle conserve les informations sur le joueur courant, contrôle l'alternance des tours, détecte les situations d'échec et de fin de partie, et permet éventuellement la sauvegarde ou le chargement d'une partie. Elle est est donc essentielle au bon fonctionnement du jeu.

Ensuite, la classe **Piece** est une classe abstraite qui regroupe les attributs communs à toutes les pièces, tels que :
- La couleur du joueur (Rouge ou Noir).
- La position actuelle sur le plateau (coordonnées x, y).

Elle définit également une méthode abstraite `estDeplacementValide()` qui sera redéfinie dans chaque classe spécifique de pièce pour appliquer les règles de mouvement propres.

Chaque pièce du Xiangqi est représentée par une sous-classe concrète :
- **General** : Représente le général (roi) de chaque camp, avec un déplacement limité au palais.
- **Garde** : Se déplace uniquement à l'intérieur du palais en diagonale.
- **Elephant** : Se déplace en deux cases en diagonale sans traverser la rivière.
- **Cheval** : Se déplace en forme de L (similaire au cavalier occidental).
- **Canon** : Se déplace comme une tour, mais capture en sautant au-dessus d'une autre pièce.
- **Tour** : Se déplace horizontalement ou verticalement sans limite de distance.
- **Soldat** : Se déplace verticalement, et latéralement après avoir traversé la rivière.

3.3 IHM Graphique

3.3.1 Conception IHM graphique

Fenêtre d'accueil Lors du lancement du logiciel, la fenêtre d'accueil et la première fenêtre que l'utilisateur voit. Cette fenêtre invite le joueur à choisir son mode de jeu via deux boutons distincts :

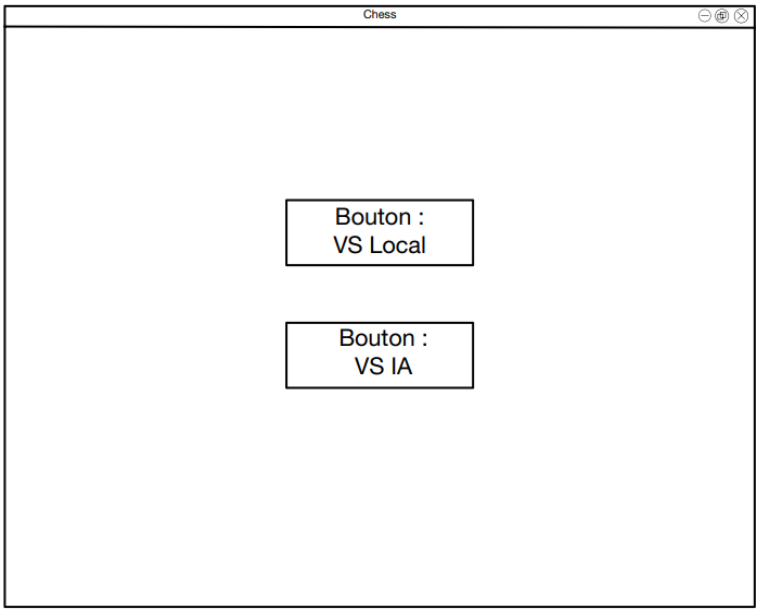


FIGURE 4 – Fenêtre de choix de mode de jeu

— Un bouton **VS Local** pour jouer à deux joueurs contre un autre joueur.
— Un bouton **VS IA** pour jouer contre une intelligence artificielle (difficulté de jeu modérée).

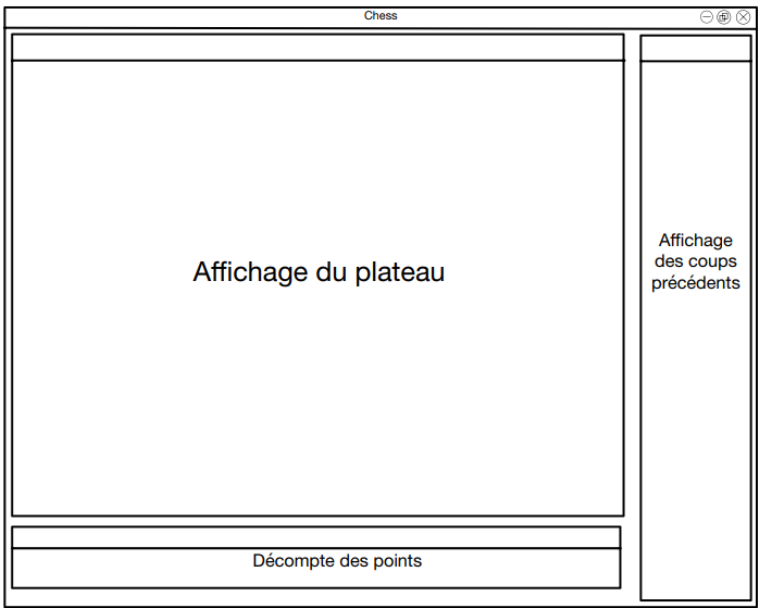


FIGURE 5 – Fenêtre de jeu

Fenêtre de jeu Lorsque le joueur choisit son mode de jeu, peu importe le mode (joueur contre joueur ou joueur contre IA), la fenêtre de jeu principale s'affiche. Cette fenêtre est divisée en plusieurs zones fonctionnelles :

Tout d'abord, au centre de l'interface, se trouve le **plateau de jeu**, représentant le terrain de Xiangqi. Le plateau est constitué d'une grille de 11 colonnes par 11 lignes, et les pièces sont placées automatiquement sur leurs positions de départ conformément aux règles du jeu. Chaque case du plateau est interactive : en cliquant sur une pièce, les déplacements possibles sont mis en évidence visuellement

en vert, et le joueur peut sélectionner une destination valide.

Sur le côté droit de l'interface, une zone est dédiée à l'**historique des coups**. Cette section affiche la liste chronologique des mouvements effectués par les joueurs, sous forme de notations simplifiées (ex : `Si le soldat se déplace de C3 à C4, il sera affiché dans la sous fenêtre : C3C4`). Cela permet aux joueurs de suivre facilement le déroulement de la partie et d'analyser les échanges de pièces.

En bas de l'interface, une autre zone affiche le **décompte des points**. Il y est affiché le tour du joueur actuel.

L'ensemble de l'interface graphique est conçu pour être clair, intuitif et fluide, afin de garantir une expérience utilisateur agréable aussi bien pour les débutants que pour les joueurs expérimentés.

3.3.2 Conception technique de l'IHM graphique

L'interface graphique du jeu a été réalisée à l'aide de la bibliothèque **Swing** de Java. L'objectif principal est de proposer une interface à la fois claire, intuitive et adaptée aux spécificités du Xiangqi.

La fenêtre principale (`JFrame`) est divisée en plusieurs zones distinctes à l'aide d'un `BorderLayout` :
- **Zone centrale** : elle accueille le **plateau de jeu**, un `JPanel` structuré en grille (`GridLayout`) de 11 colonnes et 11 lignes. Chaque case du plateau est un bouton ou une étiquette interactive permettant d'afficher une pièce et de gérer les déplacements par clic.
- **Zone à droite** : elle contient deux sections superposées :
  - **Historique des coups** : un `JTextArea` placé dans un `JScrollPane` qui liste l'ensemble des coups réalisés pendant la partie sous forme textuelle.
  - **Décompte des points** : un petit `JPanel` dédié affichant les scores des deux joueurs, mis à jour après chaque capture.

Le plateau est généré dynamiquement à l'initialisation de la partie, en créant un composant graphique pour chaque case. L'affichage des pièces utilise des icônes personnalisées, chargées depuis des fichiers image selon le type et la couleur de la pièce.

Lorsqu'un joueur clique sur une pièce, les déplacements valides sont mis en évidence (en colorant la case valide pour le déplacement en vert). En cliquant ensuite sur une case valide, la pièce se déplace, et l'état du plateau ainsi que l'historique sont mis à jour automatiquement.

L'interface a été pensée pour être réactive et légère. Chaque action est capturée par des `ActionListener` associés aux boutons du plateau.

Ainsi, l'architecture graphique respecte une séparation claire entre la logique du jeu (modèle) et son affichage (vue), selon le principe du design pattern **MVC** (Model-View-Controller) simplifié.

3.4 Conception des traitements (processus)

Cette section décrit les principaux processus métier du logiciel de Xiangqi. Chaque traitement important est modélisé par des méthodes spécifiques pour garantir la bonne application des règles et une expérience utilisateur fluide.

3.4.1 Traitement du déplacement d'une pièce

Le déplacement d'une pièce sur le plateau est réalisé grâce à la méthode `effectuerMouvement(Piece piece, int xDepart, int yDepart, int xArrivee, int yArrivee, Plateau plateau)`.

Cette méthode suit plusieurs étapes strictes :

- Vérifier que la pièce sélectionnée est valide et que la partie n'est pas terminée.
- Vérifier si le mouvement est conforme aux règles spécifiques à la pièce (appel à `mouvementValide`).
- Vérifier qu'une pièce alliée n'occupe pas la destination.
- Simuler le déplacement pour vérifier que le général du joueur ne serait pas mis en échec.
- Exécuter le mouvement si toutes les conditions sont respectées (appel à `deplacerPiece`).
- Analyser ensuite la situation pour détecter un échec ou un échec et mat.

Voici un extrait de la méthode :

Listing 1 – Méthode d'exécution d'un mouvement

```
    public static boolean effectuerMouvement(Piece piece, int xDepart, int
    yDepart, int xArrivee, int yArrivee, Plateau plateau) {
        if (piece == null) {
            System.out.println("Aucune pi ce   d placer.");
            return false;
        }
        if (plateau.isPartieTerminee()) {
            System.out.println("La partie est d j  termin e.");
            return false;
        }
        if (!piece.mouvementValide(xArrivee, yArrivee, plateau.getCases())) {
            System.out.println("Mouvement invalide.");
            return false;
        }
        Piece destination = plateau.getPiece(xArrivee, yArrivee);
        if (destination != null && destination.getCouleur() == piece.
        getCouleur()) {
            System.out.println("Impossible de capturer une pi ce alli e.");
            return false;
        }

        Piece captureTemp = plateau.getPiece(xArrivee, yArrivee);
        plateau.setPiece(xArrivee, yArrivee, piece);
        plateau.setPiece(xDepart, yDepart, null);
        boolean metEnEchec = plateau.estEnEchec(piece.getCouleur());
        plateau.setPiece(xDepart, yDepart, piece);
        plateau.setPiece(xArrivee, yArrivee, captureTemp);
        if (metEnEchec) {
            System.out.println("Mouvement annule : le general serait en
            echec !");
            return false;
        }

        plateau.deplacerPiece(xDepart, yDepart, xArrivee, yArrivee);

        Color couleurAdverse = (piece.getCouleur() == Color.RED) ? Color.
        BLACK : Color.RED;
        String nom = (couleurAdverse == Color.RED) ? "Rouge" : "Noir";

        if (plateau.estEchecEtMat(couleurAdverse)) {
            JOptionPane.showMessageDialog(
                null,
                " chec et mat au g n ral " + nom + " !",
                " chec et mat",
                JOptionPane.INFORMATION_MESSAGE
            );
            plateau.setPartieTerminee(true);
        }
```

```
        else if (plateau.estEnEchec(couleurAdverse)) {
            JOptionPane.showMessageDialog(
                null,
                " chec au g n ral " + nom + " !",
                " chec ",
                JOptionPane.WARNING_MESSAGE
            );
        }

        return true;
    }
```

Ce processus protège contre les mouvements invalides ou dangereux pour le joueur.

3.4.2 Validation spécifique des mouvements

Chaque pièce a un comportement de déplacement propre. La validation du mouvement se fait via la méthode `validerMouvement(Piece piece, int xDepart, int yDepart, int xArrivee, int yArrivee, Piece[][] plateau)`.

Selon le type de pièce, la méthode délègue la vérification :
- `validerMouvementSoldat` pour les soldats.
- `validerMouvementTour` pour les tours.
- `validerMouvementElephant` pour les éléphants, etc.

Exemple pour un éléphant :

Listing 2 – Validation du mouvement de l'éléphant

```
private static boolean validerMouvementElephant(Elephant piece, int xDepart,
int yDepart, int xArrivee, int yArrivee, Piece[][] plateau) {
    int[][] casesValidesNoir = {
        {2, 0}, {2, 4}, {8, 0}, {8, 4}, {0, 2}, {5, 2}, {10, 2}
    };

    int[][] casesValidesRouge = {
        {2, 10}, {2, 6}, {8, 10}, {8, 6}, {0, 8}, {5, 8}, {10, 8}
    };

    int[][] casesValides;
    if (piece.getCouleur() == Color.RED) {
        casesValides = casesValidesRouge;
    } else {
        casesValides = casesValidesNoir;
    }

    boolean destinationValide = false;
    for (int[] caseValide : casesValides) {
        if (caseValide[0] == xArrivee && caseValide[1] == yArrivee) {
            destinationValide = true;
            break;
        }
    }

    if (!destinationValide) {
        return false;
    }

    if (Math.abs(xArrivee - xDepart) > 3 || Math.abs(yArrivee - yDepart) >
```

```
31      {
32          return false;
33      }
34
35      return true;
36  }
```

Chaque type de pièce implémente ainsi ses propres contraintes tout en respectant les règles du Xiangqi.

3.4.3 Traitement du calcul du meilleur coup

Le calcul du meilleur coup est effectué par la méthode `calculerMeilleurCoup`, qui évalue tous les coups possibles d'un joueur et sélectionne celui qui optimise la situation sur le plateau. Ce processus inclut plusieurs étapes pour évaluer chaque coup en fonction de la position des pièces, des captures possibles, des coups légaux et des menaces.

Le processus détaillé est le suivant :
- Parcourir toutes les pièces du joueur.
- Pour chaque pièce, analyser tous les coups possibles de celle-ci sur le plateau (coordonnées de départ et d'arrivée).
- Vérifier que le coup est légal.
- Calculer un score pour chaque coup, en prenant en compte :
  - La valeur de la pièce capturée.
  - Des bonus spécifiques pour certaines pièces comme le soldat.
  - Le nombre de coups légaux possibles après ce mouvement.
  - Le nombre de menaces sur la pièce après le mouvement.
- Si le coup génère un score plus élevé que le précédent meilleur coup, le mettre à jour.

Voici un extrait de la méthode `calculerMeilleurCoup` :

Listing 3 – Calcul du meilleur coup

```
1  public int[] calculerMeilleurCoup() {
2      int meilleurScore = Integer.MIN_VALUE;
3      int[] meilleurCoup = null;
4
5      for (Piece p : getPieces(couleur)) {
6          int x0 = p.getX(), y0 = p.getY();
7          for (int x1 = 0; x1 < 11; x1++) {
8              for (int y1 = 0; y1 < 11; y1++) {
9                  if (!isLegalMove(p, x1, y1)) continue;
10
11                 Piece cible = plateau.getPiece(x1, y1);
12                 plateau.deplacerPiece(x0, y0, x1, y1);
13
14                 int score = 0;
15                 if (cible != null) {
16                     score += 10 * VAL_PIECE.get(cible.getClass());
17                 }
18                 if (p instanceof Soldat) {
19                     int dy = (couleur == Color.BLACK) ? (y1 - y0) : (y0 - y1);
20                     if (dy > 0) score += 2 * dy;
21                 }
22                 score += compterCoupsLegaux(p, x1, y1);
23                 score += 3 * compterMenaces(p, x1, y1);
24
25                 plateau.deplacerPiece(x1, y1, x0, y0);
```

```
                    plateau.setPiece(x1, y1, cible);

                    if (score > meilleurScore) {
                        meilleurScore = score;
                        meilleurCoup = new int[]{x0, y0, x1, y1};
                    }
                }
            }
        }
        return meilleurCoup;
    }
```

3.4.4 Gestion de la capture d'une pièce

Lorsqu'une pièce arrive sur une case occupée par une pièce adverse, la capture est gérée par `deplacerPiece(int x, int y, int newX, int newY)`.

Le processus est le suivant :

- Sauvegarde de la pièce capturée.
- Vérification que le mouvement ne met pas son propre général en échec.
- Suppression de la pièce adverse.
- Déplacement de la pièce active.
- Mise à jour des coordonnées de la pièce.

Voici un extrait :

Listing 4 – Méthode de déplacement d'une pièce

```
    public void deplacerPiece(int x, int y, int newX, int newY) {
        if (partieTerminee) {
            afficherMessageFinPartie("La␣partie␣est␣d j ␣termin e␣!");
            return;
        }

        Piece piece = getPiece(x, y);
        Piece pieceCapturee = getPiece(newX, newY);

        if (piece != null) {
            if (metLeGeneralEnEchec(piece, x, y, newX, newY)) {
                setPiece(newX, newY, pieceCapturee);
                System.out.println("Impossible␣de␣d placer␣la␣pi ce ,␣le
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣g n ral␣serait␣en␣ chec ␣!");
                return;
            }

            setPiece(newX, newY, piece);
            piece.setPosition(newX, newY);
            setPiece(x, y, null);

            verifierFinDePartie(piece, x, y, newX, newY);
        }
    }
```

Cela assure une capture sécurisée sans enfreindre les règles du jeu.

3.4.5 Détection de la fin de partie

Après chaque mouvement, la méthode `verifierFinDePartie` est appelée pour vérifier si la partie doit être arrêtée.

Cette méthode examine plusieurs conditions permettant de détecter la fin de la partie :
- `generalFaceAFace` : Vérifie si les généraux des deux joueurs sont face à face, ce qui constitue une situation de fin de partie immédiate.
- `estEchecEtMat` : Vérifie si un joueur est en situation d'échec et mat, ce qui signifie qu'il a perdu la partie.
- `estPat` : Vérifie si la partie est en situation de pat, c'est-à-dire qu'aucun joueur ne peut jouer sans enfreindre les règles, mais sans que le jeu soit terminé par échec et mat.

En cas de détection de l'une de ces situations :
- Un message est affiché à l'utilisateur grâce à la méthode `afficherMessageFinPartie`, qui informe de la fin de la partie, en fonction de la condition détectée.
- L'état `partieTerminee` est mis à `true`, signalant que la partie est terminée et empêchant de nouveaux mouvements.

Voici le code correspondant :

Listing 5 – Méthode de détection de la fin de partie

```
1     // Methode pour verifier si la partie doit etre arretee
2     private void verifierFinDePartie(Piece piece, int x, int y, int newX, int
3     newY) {
4         if (generalFaceAFace()) {
5             afficherMessageFinPartie("Les generaux sont face a face ! La
6 partie est terminee !");
7             partieTerminee = true;
8         } else if (estEchecEtMat(piece.getCouleur())) {
9             afficherMessageFinPartie("La partie est terminee : chec et
10 Mat !");
11             partieTerminee = true;
12         } else if (estPat(piece.getCouleur())) {
13             afficherMessageFinPartie("La partie est terminee : Pat !");
14             partieTerminee = true;
15         }
16     }
```

Le fonctionnement de cette méthode est le suivant :
- Elle est appelée après chaque mouvement pour vérifier si une condition de fin de partie est remplie.
- Elle commence par vérifier si les généraux sont face à face à l'aide de la méthode `generalFaceAFace`. Si cette condition est vraie, la méthode affiche un message indiquant que la partie est terminée et met l'état `partieTerminee` à `true`.
- Si ce n'est pas le cas, elle vérifie ensuite si la partie est en situation d'échec et mat avec `estEchecEtMat`. En cas de détection, un message est affiché et l'état de la partie est mis à `true`.
- Enfin, si ni l'une ni l'autre des conditions précédentes n'est remplie, la méthode vérifie si la partie est en situation de pat avec `estPat`. Si c'est le cas, un message est également affiché, et la partie est marquée comme terminée.

Ce processus garantit que la fin de la partie est détectée de manière fiable et qu'un message clair est affiché pour informer l'utilisateur, tout en mettant à jour l'état de la partie pour l'empêcher de continuer.

4 Manuel utilisateur

Cette section présente un guide d'utilisation du logiciel de Xiangqi. Elle explique comment démarrer une partie, interagir avec l'interface, et comprendre les principales fonctionnalités. Toutes les étapes sont accompagnées de captures d'écran illustratives numérotées et commentées.

4.1 Démarrage et choix du mode de jeu

Lors de l'exécution du programme, une fenêtre d'accueil (Figure 6) s'affiche automatiquement. L'utilisateur se voit proposer deux options principales :

— **Joueur contre Joueur (local)** : Deux joueurs humains jouent l'un contre l'autre sur le même ordinateur.

— **Joueur contre IA** : Un joueur humain affronte une intelligence artificielle intégrée.

L'utilisateur sélectionne son mode en cliquant sur le bouton correspondant.



FIGURE 6 – Fenêtre d'accueil du logiciel

Après avoir choisi le mode de jeu, le logiciel instancie une nouvelle partie et charge l'interface principale.

4.2 Plateau de jeu

La fenêtre de jeu principale est composée d'un plateau représentant fidèlement les dimensions officielles du Xiangqi : 11 colonnes et 11 lignes (voir Figure 7). Les pièces sont positionnées conformément à la configuration standard initiale. Chaque pièce est représentée par une icône claire et facilement reconnaissable selon sa couleur (Rouge ou Noir) et son type (Général, Garde, Éléphant, etc.).

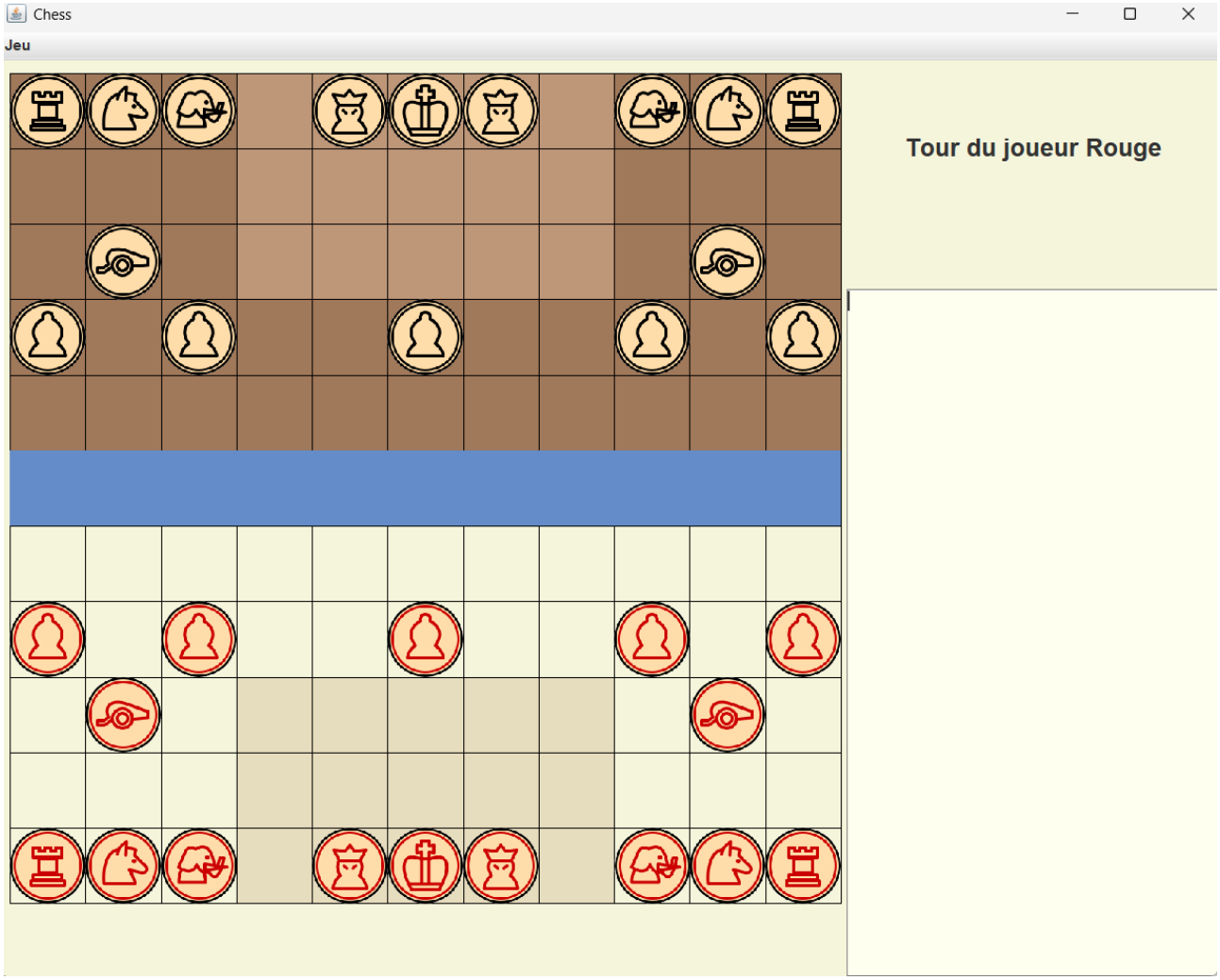


FIGURE 7 – Plateau de Xiangqi à l'initialisation

Le joueur peut cliquer sur une pièce pour la sélectionner. Les déplacements légaux s'affichent alors visuellement. Un second clic sur une case valide permet d'effectuer le déplacement. Cela permet au joueur de mieux visualiser ses options de déplacement.

4.3 Sélectionner et déplacer une pièce

Pour déplacer une pièce, l'utilisateur doit cliquer une première fois sur la pièce souhaitée, puis cliquer une seconde fois sur l'une des cases autorisées (voir Figure 8). Si la destination est valide selon les règles du Xiangqi, la pièce est automatiquement déplacée, sinon le logiciel affiche un message d'erreur.

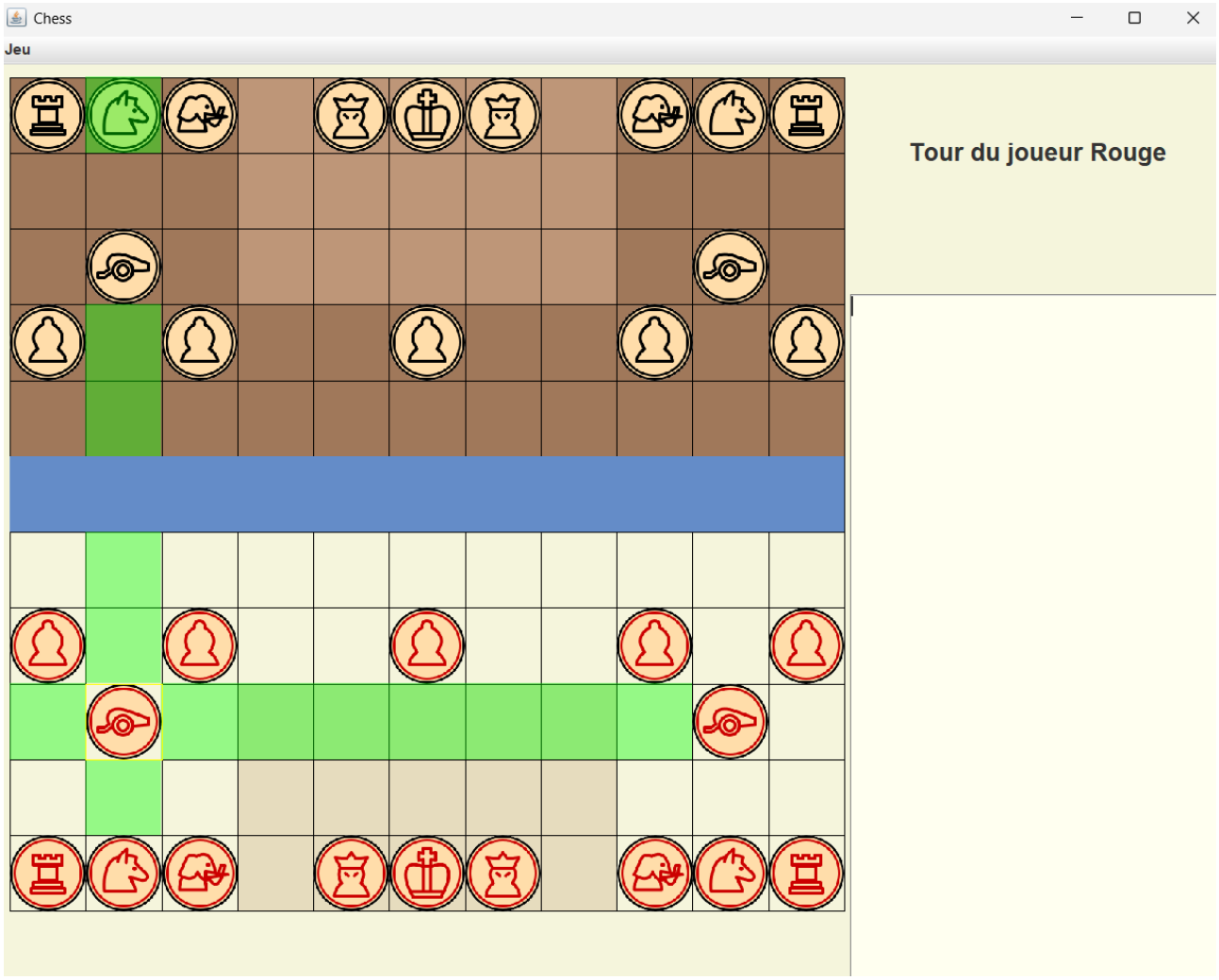


FIGURE 8 – Illustration de mouvement valide

Lorsqu'une pièce se déplace sur une case occupée par une pièce adverse, celle-ci est capturée et retirée du plateau.

4.4 Historique des coups

paragraph Sur la partie droite de l'écran, l'utilisateur dispose d'un panneau intitulé « Historique des coups » (voir Figure 9). Ce panneau liste l'ensemble des coups joués dans l'ordre chronologique, en indiquant les pièces déplacées et leurs positions de départ et d'arrivée.

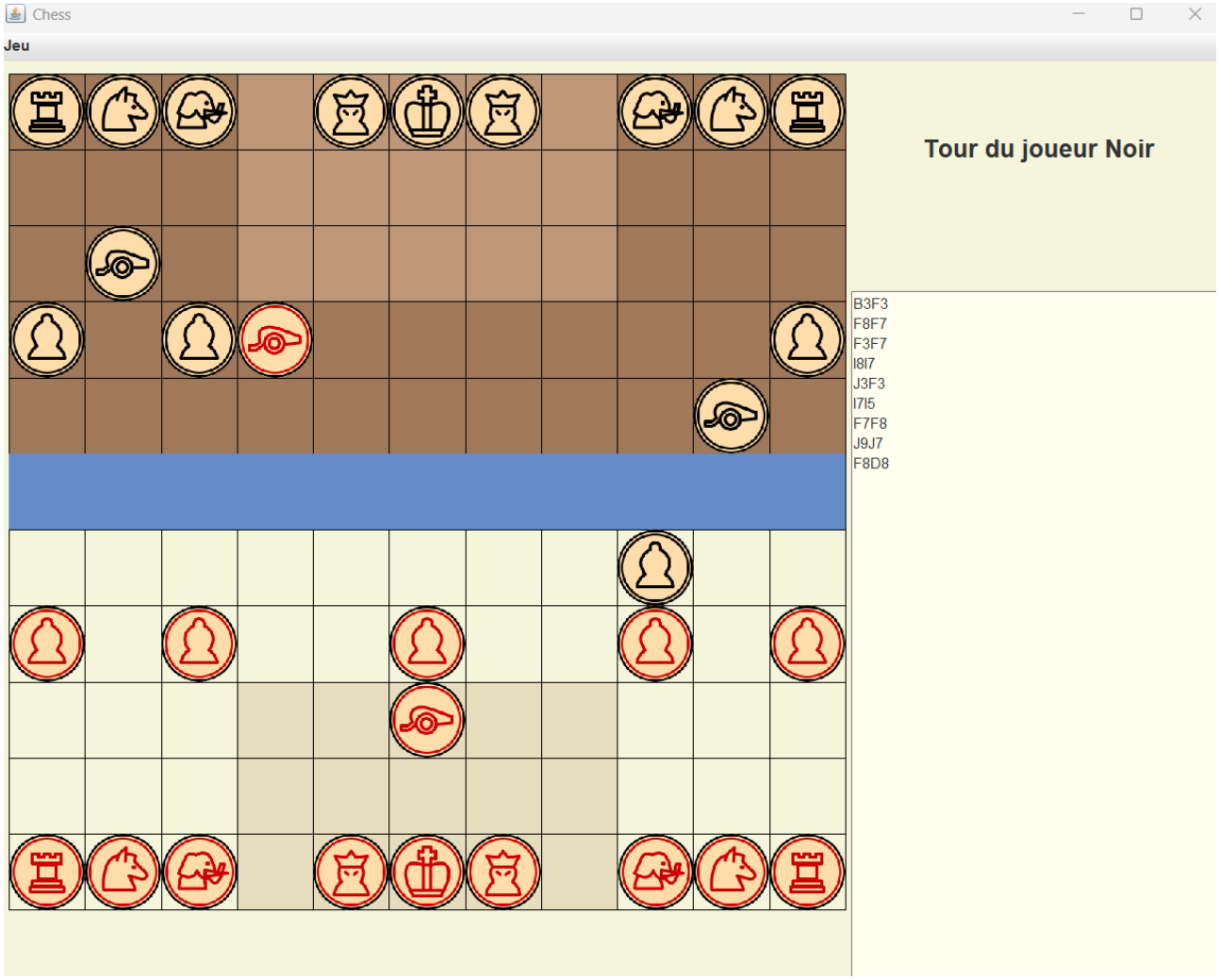


FIGURE 9 – Affichage de l'historique des coups

Cela permet aux joueurs de suivre l'évolution de la partie, d'analyser les coups stratégiques ou de revenir sur des erreurs éventuelles.

4.5 Fin de la partie

La partie peut se terminer dans les cas suivants :

— Le général d'un des deux camps est capturé (échec et mat).

— Aucun déplacement légal n'est possible (pat).

Lorsqu'une condition de fin est atteinte, une boîte de dialogue (Figure 10) s'affiche pour annoncer le vainqueur de la partie ou signaler une égalité.

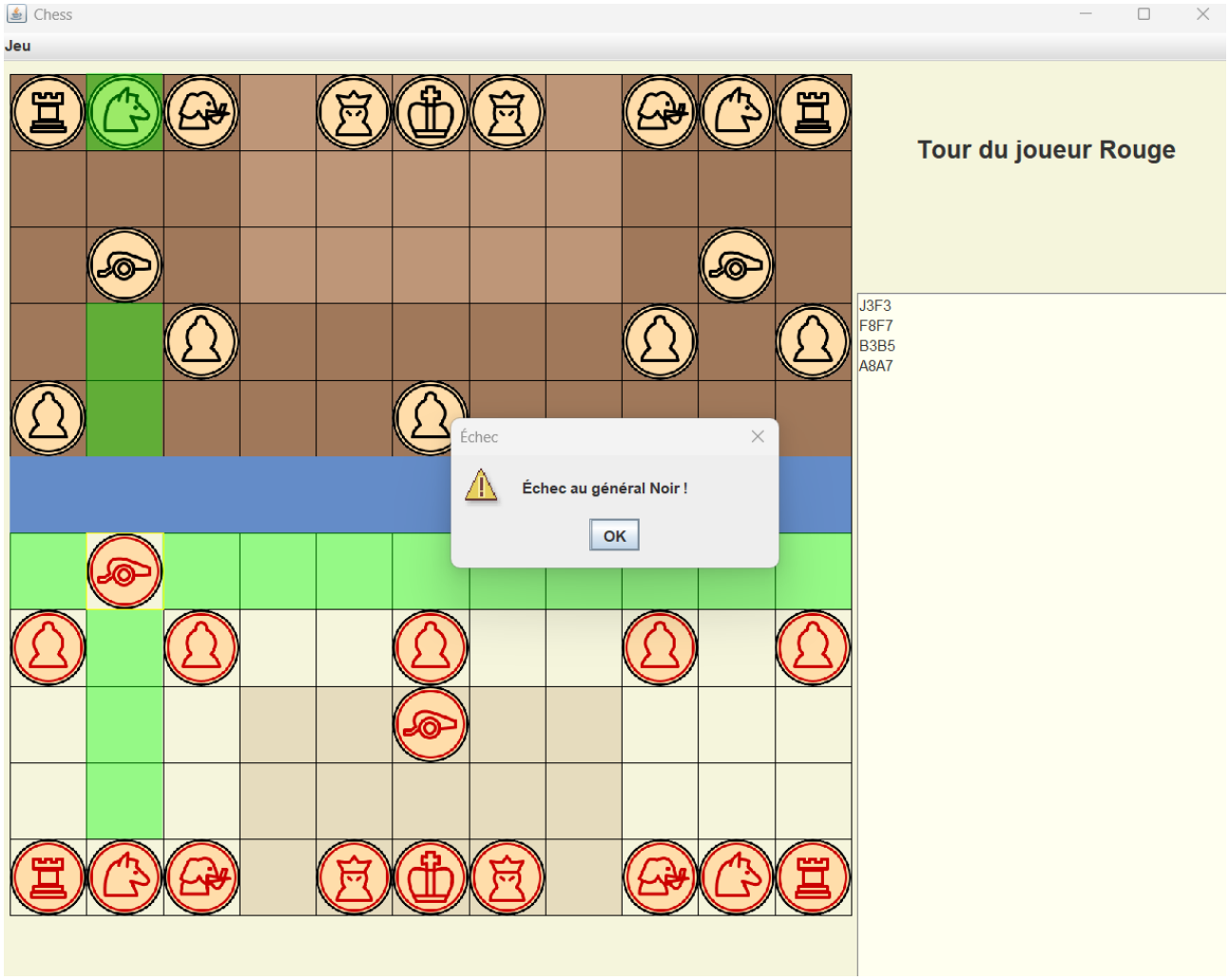


FIGURE 10 – Message de fin de partie

5 Déroulement du projet

Dans cette section, nous décrivons comment le projet a été réalisé en équipe : la répartition des tâches, la synchronisation du travail en membres de l'équipe, etc.

5.1 Réalisation du projet par étapes

Cette figure 11 présente les tâches majeurs de la réalisation du logiciel ainsi que leur répartition.

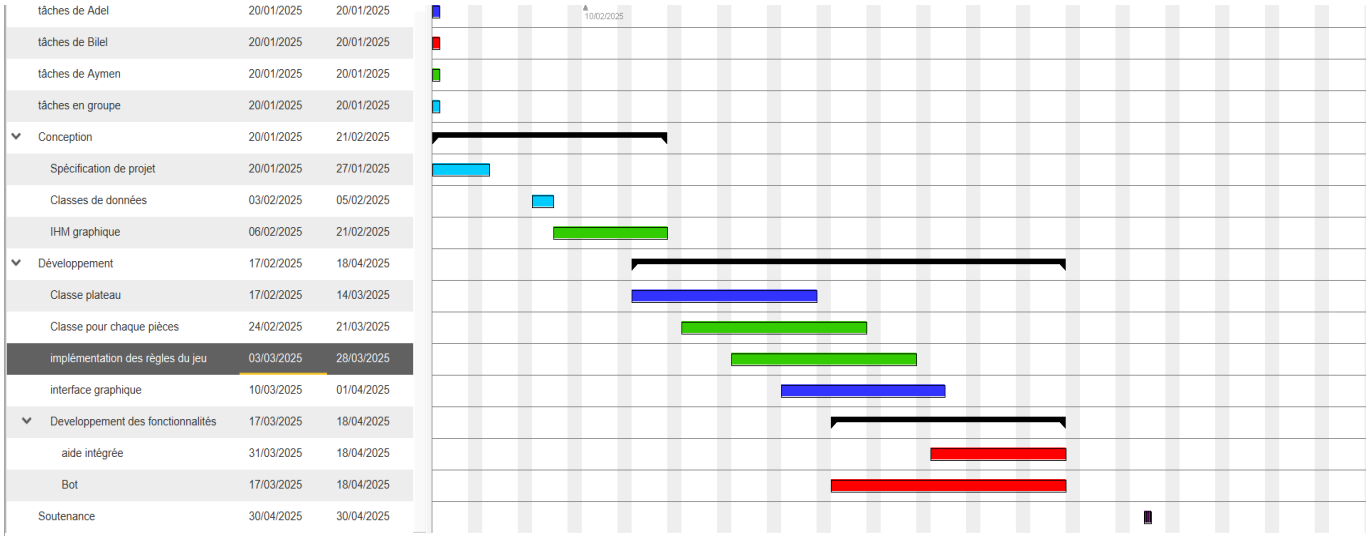


FIGURE 11 – Diagramme de Gantt

5.2 Répartition des tâches, communication et intégration

Afin d'assurer une progression efficace du projet, nous avons organisé la répartition des tâches de manière structurée et progressive. Dès le lancement du projet, chaque membre de l'équipe (Adel, Bilel et Aymen) s'est vu attribuer des responsabilités spécifiques tout en participant aux tâches transversales en groupe. La phase de conception, débutée en janvier 2025, a été collective pour garantir une compréhension commune des règles du jeu et des choix techniques. Ensuite, la réalisation des classes de données et l'élaboration de l'IHM graphique ont été réparties de façon à optimiser le temps et à permettre un travail en parallèle.

Le développement proprement dit a été divisé par modules fonctionnels : la création du plateau, l'implémentation des classes de pièces, les règles de déplacement et la construction de l'interface graphique ont avancé en parallèle entre février et avril. Le développement des fonctionnalités complémentaires, telles que l'aide intégrée et le bot (joueur automatique), a été engagé dès la stabilisation du cœur du projet. Chaque fonctionnalité majeure était prise en charge par un ou deux membres, puis intégrée progressivement au projet principal.

La communication au sein de l'équipe a été régulière et fluide, grâce à l'utilisation d'outils de collaboration comme GitHub pour le partage de code et la gestion des versions. Des réunions hebdomadaires nous ont permis de faire le point sur l'avancement, de résoudre rapidement les blocages techniques et de synchroniser l'intégration des modules. Enfin, une phase de finalisation commune a été prévue avant la soutenance, afin d'assurer la cohérence de l'ensemble du logiciel et de peaufiner les derniers détails fonctionnels et esthétiques.

— **Répartition des rôles** : Chaque membre (Adel, Bilel, Aymen) s'est vu attribuer des responsabilités spécifiques (classes de données, IHM graphique, développement des fonctionnalités) tout en participant aux tâches communes.

— **Travail en groupe sur la conception** : La phase de conception du projet (spécifications, modélisation des classes, choix techniques) a été réalisée collectivement pour garantir une compréhension et une cohérence globales.

— **Développement modulaire** :
  — Développement du plateau de jeu et des classes de pièces.
  — Implémentation des règles de déplacement et des conditions de victoire.
  — Construction de l'interface graphique (Swing).
  — Ajout des fonctionnalités secondaires : aide intégrée, sauvegarde/chargement, bot.

— **Communication et collaboration** :
  — Réunions régulières pour faire le point sur l'avancement.
  — Utilisation de GitHub pour le versionnage, la fusion du code et la collaboration.
— **Phase d'intégration et de finalisation** :
  — Intégration progressive des modules développés.
  — Tests fonctionnels et corrections des bugs.
  — Nettoyage du code, finalisation de l'interface et rédaction du rapport en LaTeX.
— **Soutenance** : Préparation collective de la présentation finale prévue pour le 30 avril 2025.

6 Conclusion et perspectives

Dans cette section, nous résumons la réalisation du projet et nous présentons également les extensions et améliorations possibles du projet.

6.1 Résumé du travail réalisé

Ce projet de développement d'une adaptation du jeu d'échecs chinois a été une expérience extrêmement enrichissante. Il nous a permis d'acquérir et développer nos compétences en conception logicielle, en gestion de projet et en travail d'équipe dans un cadre concret et exigeant. En choisissant de nous orienter vers une adaptation du jeu de Chess, nous avons pu explorer en profondeur les mécanismes de stratégie et de planification, tout en relevant les défis liés à la modélisation du jeu, à l'interface utilisateur et à la gestion de règles complexes.

Tout au long du projet, nous avons su organiser notre travail de manière méthodique, répartir efficacement les tâches et faire preuve d'une communication constante pour assurer la cohérence de l'ensemble. Cette expérience nous a également confrontés aux réalités du développement logiciel : l'importance de l'anticipation, de la gestion du temps, de la résolution collaborative des problèmes et de l'adaptabilité face aux imprévus.

Les différentes fonctionnalités attendues, soigneusement identifiées et pondérées, visent à offrir une expérience de jeu fidèle, interactive et accessible. L'accent a été mis sur la justesse des règles, la fluidité de l'interface graphique et la robustesse du système de validation des mouvements, tout en intégrant des éléments annexes essentiels comme la sauvegarde des parties et l'affichage de l'aide.

Notre travail s'est structuré autour de plusieurs axes forts : la modélisation rigoureuse des règles du jeu, la création d'une interface graphique intuitive avec Swing, la mise en place d'un système de validation des mouvements robuste, ainsi que l'ajout de fonctionnalités complémentaires telles que la sauvegarde des parties et l'aide intégrée. L'utilisation d'outils adaptés comme Java, Eclipse, GitHub et LaTeX nous a permis d'assurer un développement efficace et une documentation claire.

6.2 Améliorations possibles du projet

Bien que notre projet réponde aux objectifs initiaux et propose une adaptation fidèle et fonctionnelle du Xiangqi, plusieurs pistes d'amélioration pourraient être envisagées pour enrichir l'expérience utilisateur et étendre les fonctionnalités du logiciel :

Ajout d'une intelligence artificielle avancée Actuellement, le projet propose un mode multijoueur et un affrontement basique contre l'ordinateur. Une amélioration importante serait de développer une intelligence artificielle plus performante, capable d'analyser plusieurs coups à l'avance et d'adapter sa stratégie en fonction de la situation du plateau. Cela permettrait d'offrir un véritable défi aux joueurs expérimentés.

Création de différents niveaux de difficulté Pour rendre le jeu accessible à un public plus large, il serait intéressant d'introduire plusieurs niveaux de difficulté, ajustant la profondeur d'analyse de l'intelligence artificielle ainsi que la stratégie employée.

Amélioration de l'interface graphique Bien que l'interface actuelle soit fonctionnelle, une refonte graphique plus moderne et plus esthétique pourrait améliorer l'ergonomie du logiciel. L'ajout d'animations fluides pour les déplacements des pièces, d'effets visuels pour les captures et d'un retour visuel pour les erreurs de mouvement pourrait rendre l'expérience utilisateur plus agréable.

Ajout d'un mode tutoriel Un mode tutoriel guidé, expliquant progressivement les règles du Xiangqi et proposant des exercices interactifs, pourrait faciliter l'apprentissage pour les débutants et démocratiser la pratique de ce jeu.

Références

[HP25] John L. Hennessy and David A. Patterson. *Rapport Chess*. Chess Group, CYU, 5th edition, 2025.

[MAB25] Adel MAHI, ESSAHLI Aymen, and BEN NAYA Bilel. Rpport de projet. *IBM Syst. J.*, 41(4) :28–30, 2025.